

The lab assignment today is to clone this repository (CS111-jtotto/Course-Information-Fall2026) and modify the source code at Week 1/Hello.java.

1. Copy the Java source to your own file, named something like HelloMyName.java. Now it will fail to compile, because it doesn't define a class called HelloMyName.
2. Change the name of your class to match its filename. Verify that it compiles.

Send me your Hello__.java file by email by the end of today, which will count for the lab assignment.

Below are today's lecture notes. Please review them for the quiz on Wednesday, which will cover usage of the terminal and the Java compiler.

Two more steps to prep for Wednesday: - Read "Code structure in Java" through "Writing a class with a main()", to familiarize yourself with classes and their role in running a program.

https://learning.oreilly.com/library/view/head-first-java/9781492091646/ch01.html#what_goes_in_a_source_filequestion_mark - Accept the organization invite from CS111-jtotto, so that you will be able to submit labs through GitHub commits.

Finally, please note the TAs' office hours have changed. Paria's office hours now run 11:30am – 1:30pm on Thu (and not on Friday), whilst Enzo's office hours are by appointment.

Anatomy of a Command

Every terminal command follows a standard syntax:

command [flags/options] [arguments]

- **Command:** The utility or executable program you are running (e.g., ls).
- **Flags / Options:** Modifiers that change how the command behaves, often prefixed by one or two dashes.
- **Arguments:** The target that the command operates on, such as a file path, directory, or text string (e.g., documents/).

Example syntax:

```
ls -l /Users/username/Documents
```

```
# | | └ Argument (Target directory)
```

```
# | └ Flag (Long-format listing modifier)
```

```
# └ Command (List contents)
```

Core Navigation & File Commands

cd — **Change Directory** Navigates through your file system's folder hierarchy.

- `cd Documents` — Moves into the Documents folder located in your current directory.
- `cd ..` — Moves up one directory level toward the root directory.
- `cd ~` — Returns directly to your user home directory.

ls — List Directory Contents Displays files and subdirectories stored within a given directory.

- `ls` — Lists visible files in the current directory.
- `ls -a` — Lists **all** files, including hidden files (those starting with a `.`).
- `ls -lh` — Displays detailed file info (permissions, size, owner) in a human-readable format.

cp — Copy Files and Directories Duplicates files or folders from a source location to a destination.

- `cp file.txt backup.txt` — Copies `file.txt` to a new file named `backup.txt`.
- `cp -r project/project_backup/` — Recursively copies the entire project directory and its contents.

rm — Remove (Delete) Files and Directories Permanently removes files or directories from your disk without sending them to a trash bin.

- `rm file.txt` — Immediately deletes `file.txt`.
- `rm -r foldername/` — Recursively deletes a folder and everything inside it.
- `rm -rf foldername/` — **Caution:** Force-deletes a folder recursively without asking for confirmation.

System Controls & Documentation

Along with `git` commands, we'll cover these quality of life controls on Wednesday.

<Ctrl> + c — Cancel / Interrupt Signal Pressing `Ctrl` and `c` simultaneously will forcibly interrupt a running terminal program. It's useful to break out of unresponsive programs or infinite loops.

<Up Arrow> — Command History (Previous Line)

Pressing the **Up Arrow** key cycles backward through your terminal's execution history, bringing up previously entered commands one by one.

- **Re-running Commands:** Instantly recall and re-run your last executed command by pressing `<Up Arrow>` then `<Enter>`.
- **Fixing Mistakes:** Retrieve a previous long or complex command, edit typos using the arrow keys or shortcuts, and press `<Enter>` to run the corrected command.
- **Browsing History:** Continuously press `<Up Arrow>` to search further back into your session history. Use `<Down Arrow>` to navigate back toward the newest commands.

<Tab> — Tab Completion (Auto-Complete)

Pressing the **Tab** key automatically completes command names, file paths, directory names, and flags as you type. Combine it with `ls` to find files quickly through the terminal.

- **Single Press:** If there is only one matching command, file, or folder, pressing <Tab> finishes typing the name automatically and adds a space or trailing slash /.
- **Double Press (<Tab> + <Tab>):** If multiple items match what you have typed so far, pressing <Tab> twice displays a list of all matching possibilities.